

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 8

з дисципліни «Алгоритмізація та програмування»
на тему "Реалізація алгоритмів сортування та робота з файлами на мові C ++"

XAI.301.G5.310.4 ЛР

Виконав студент гр. _____310_____

17.05.2026

(підпис, дата)

Дмитро ДОБРОВЕЦЬКИЙ

(П.І.Б.)

Перевірів

_____к.т.н., доц. Олена ГАВРИЛЕНКО

(підпис, дата)

(П.І.Б.)

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt».

Array98. Дан цілочисельний масив розміру N. Видалити з масиву всі елементи, що зустрічаються менше трьох разів, і вивести розмір отриманого масиву і його вміст.

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n – номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Matrix20. Дана матриця розміру $M \times N$. Для кожного стовпця матриці знайти добуток його елементів.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань - за кодом, що є у файлі звіту і теоретичними відомостями у файлі лекції»

«Оціни повноту, правильність, час відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Сформуль загальну оцінку у 10- бальній шкалі, віднімаючи 50% балів там, де ймовірність відповіді з засобом ШІ висока»

Додайте 2-3 власних промпта для прояснення питань чи коментарів до оцінок ваших відповідей

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Array98

Вхідні дані (ім'я, опис, тип, обмеження):

- **N** – початкова кількість елементів у масиві, цілий тип (int);
- **arr[i]** – елементи початкового цілочисельного масиву, цілий тип (int).

Вихідні дані (ім'я, опис, тип):

- **N** – новий (змінений) розмір масиву після видалення елементів, цілий тип (int);
- **arr[i]** – вміст отриманого масиву після видалення елементів, що зустрічалися менше 3 разів, цілий тип (int).

Лістинг коду вирішення задачі Array98 наведено в дод. А (стор. 4).

Екрани робіт програми показаний на рис. Б.1.

Завдання 2.

Вирішення задачі Matrix20

Вхідні дані (ім'я, опис, тип, обмеження):

- **m** – кількість рядків матриці, цілий тип, $2 \leq m \leq 50$;
- **n** – кількість стовпців матриці, цілий тип, $2 \leq n \leq 50$;
- **matr[i][j]** – елементи матриці, цілий тип;

- **filename** – ім'я файлу для зчитування/запису матриці, текстовий тип.

Вихідні дані (ім'я, опис, тип):

- **Консольне виведення** – добуток елементів для кожного стовпця матриці, цілий тип;
- **filename** – текстовий файл із дозаписаними результатами обчислень.

Лістинг коду вирішення задачі Matrix20 наведено в дод. А (стор. 4).

Екрани робіт програми показаний на рис. Б.1.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

Завдання 5.

Діалог з Chat GPT для самоаналізу наведено у дод. В.

ВИСНОВКИ

У ході виконання роботи було вивчено основні алгоритми обробки одновимірних і двовимірних масивів мовою C++, а також засвоєно принципи роботи з файлами за допомогою стандартних бібліотек. Реалізовано оголошення масивів, зчитування даних з файлу, їх обробку відповідно до поставлених завдань та запис результатів у вихідний файл у середовищі Visual Studio. Отримані навички дозволяють ефективно працювати з масивами та файловим введенням/виведенням під час розробки програм.

ДОДАТОК А

Лістинг коду програми

main.cpp

```
#include <iostream>
#include <string>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

void task1();
void task2();
void task3();

int main()
{
    int choice;

    do
    {
        cout << "\n1 - Завдання 1";
        cout << "\n2 - Завдання 2";
        cout << "\n3 - Завдання 3";
        cout << "\n0 - Вихід";
        cout << "\nВаш вибір: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                task1();
                break;
            case 2:
                task2();
                break;
            case 3:
                task3();
                break;
            case 0:
                cout << "Вихід.\n";
                break;
            default:
                cout << "Помилка!\n";
        }

    } while (choice != 0);

    return 0;
}
```

```
void task1()
{
    string inFile, outFile;
    int arr[MAX], n;

    cout << "Введіть ім'я вхідного файлу: ";
    cin >> inFile;

    cout << "Введіть ім'я вихідного файлу: ";
    cin >> outFile;

    getArray(inFile, arr, n);

    cout << "\nПочатковий масив:\n";
    printArray(arr, n);

    removeLessThanThree(arr, n);

    cout << "\nРезультат:\n";
    cout << "Новий розмір: " << n << endl;
    printArray(arr, n);

    saveArray(outFile, arr, n);
}

void task2()
{
    string fileName;
    int matrix[MAX][MAX], m, n;

    cout << "Введіть ім'я файлу: ";
    cin >> fileName;

    getMatrix(fileName, matrix, m, n);

    cout << "\nМатриця:\n";
    printMatrix(matrix, m, n);

    columnProducts(matrix, m, n);

    appendResult(fileName, matrix, m, n);
}

void task3()
{
    string inFile, outFile;
    char arr[MAX];
    int n;
```

```

    cout << "Введіть ім'я вхідного файлу: ";
    cin >> inFile;

    cout << "Введіть ім'я вихідного файлу: ";
    cin >> outFile;

    getCharArray(inFile, arr, n);

    doubleSelectionSortDesc(arr, n);

    cout << "\nВідсортований масив:\n";
    printCharArray(arr, n);

    saveCharArray(outFile, arr, n);
}

                                array_utils.h
#ifndef MATRIX_UTILS_H
#define MATRIX_UTILS_H
#include "array_utils.h"

#include <string>
using namespace std;

void getMatrix(string filename, int matrix[][MAX], int& m, int& n);
void printMatrix(int matrix[][MAX], int m, int n);
void columnProducts(int matrix[][MAX], int m, int n);
void appendResult(string filename, int matrix[][MAX], int m, int n);

#endif

                                array_utils.cpp
#include "array_utils.h"
#include <fstream>
#include <iostream>

using namespace std;

void getArray(string filename, int arr[], int& n)
{
    ifstream fin(filename);

    fin >> n;
    for (int i = 0; i < n; i++)
        fin >> arr[i];

    fin.close();
}

void saveArray(string filename, const int arr[], int n)
{

```

```

    ofstream fout(filename);

    fout << n << endl;
    for (int i = 0; i < n; i++)
        fout << arr[i] << " ";

    fout.close();
}

void printArray(const int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

void removeLessThanThree(int arr[], int& n)
{
    for (int i = 0; i < n; i++)
    {
        int count = 0;

        for (int j = 0; j < n; j++)
            if (arr[i] == arr[j])
                count++;

        if (count < 3)
        {
            for (int k = i; k < n - 1; k++)
                arr[k] = arr[k + 1];

            n--;
            i--;
        }
    }
}

void getCharArray(string filename, char arr[], int& n)
{
    ifstream fin(filename);

    fin >> n;
    for (int i = 0; i < n; i++)
        fin >> arr[i];

    fin.close();
}

void saveCharArray(string filename, const char arr[], int n)
{

```

```

    ofstream fout(filename);

    fout << n << endl;
    for (int i = 0; i < n; i++)
        fout << arr[i] << " ";

    fout.close();
}

void printCharArray(const char arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

void doubleSelectionSortDesc(char arr[], int n)
{
    int left = 0, right = n - 1;

    while (left < right)
    {
        int maxIndex = left;
        int minIndex = left;

        for (int i = left; i <= right; i++)
        {
            if (arr[i] > arr[maxIndex])
                maxIndex = i;

            if (arr[i] < arr[minIndex])
                minIndex = i;
        }

        swap(arr[left], arr[maxIndex]);

        if (minIndex == left)
            minIndex = maxIndex;

        swap(arr[right], arr[minIndex]);

        left++;
        right--;
    }
}

matrix_utils.h

#ifndef MATRIX_UTILS_H
#define MATRIX_UTILS_H
#include "array_utils.h"

```

```

#include <string>
using namespace std;

void getMatrix(string filename, int matrix[][MAX], int& m, int& n);
void printMatrix(int matrix[][MAX], int m, int n);
void columnProducts(int matrix[][MAX], int m, int n);
void appendResult(string filename, int matrix[][MAX], int m, int n);

#endif

matrix_utils.cpp

#include "matrix_utils.h"
#include <fstream>
#include <iostream>

using namespace std;

void getMatrix(string filename, int matrix[][MAX], int& m, int& n)
{
    ifstream fin(filename);

    fin >> m >> n;

    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            fin >> matrix[i][j];

    fin.close();
}

void printMatrix(int matrix[][MAX], int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}

void columnProducts(int matrix[][MAX], int m, int n)
{
    cout << "\nДобутки стовпців:\n";

    for (int j = 0; j < n; j++)
    {
        int product = 1;

        for (int i = 0; i < m; i++)
            product *= matrix[i][j];
    }
}

```

```

        cout << product << " ";
    }
    cout << endl;
}

void appendResult(string filename, int matrix[][MAX], int m, int n)
{
    ofstream fout(filename, ios::app);

    fout << "\n\nДобутки стовпців:\n";

    for (int j = 0; j < n; j++)
    {
        int product = 1;

        for (int i = 0; i < m; i++)
            product *= matrix[i][j];

        fout << product << " ";
    }

    fout.close();
}

array_in_4.txt
8
1 2 2 2 3 4 4 5

matr_in_4.txt
2 3
1 2 3
4 5 6

char_in_4.txt
6
a z d k b q

```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```
1 - Завдання 1
2 - Завдання 2
3 - Завдання 3
0 - Вихід
Ваш вибір: 1
Введіть ім'я вхідного файлу: array_in_4.txt
Введіть ім'я вихідного файлу: array_out_4.txt
```

Початковий масив:

```
1 2 2 2 3 4 4 5
```

Результат:

Новий розмір: 3

```
2 2 2
```

```
1 - Завдання 1
2 - Завдання 2
3 - Завдання 3
0 - Вихід
Ваш вибір: 2
Введіть ім'я файлу: matr_in_4.txt
```

Матриця:

```
1 2 3
```

```
4 5 6
```

Добутки стовпців:

```
4 10 18
```

```
1 - Завдання 1
2 - Завдання 2
3 - Завдання 3
0 - Вихід
Ваш вибір: 3
Введіть ім'я вхідного файлу: char_in_4.txt
Введіть ім'я вихідного файлу: char_out_4.txt

Відсортований масив:
z q k d b a
```

Додаток В Діаграми активності для задач

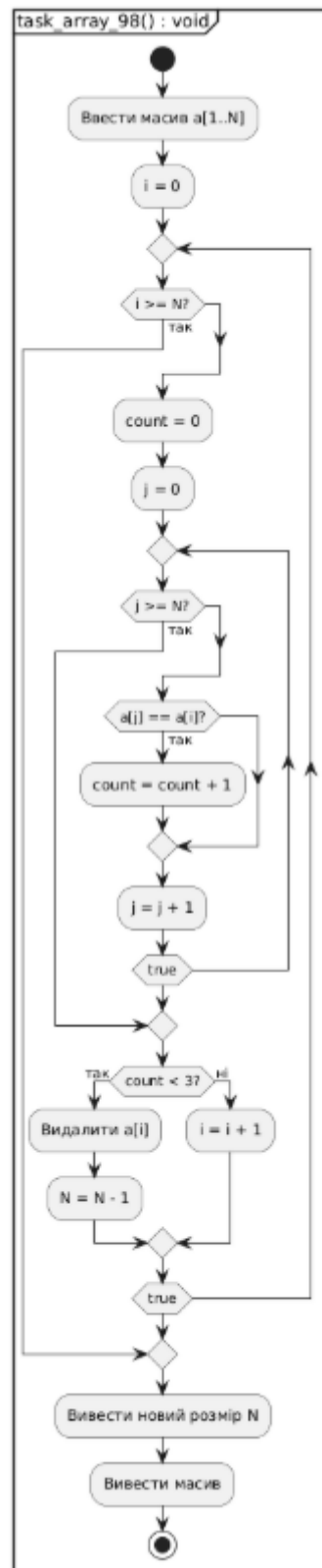


Рисунок 1 – діаграма активності для задачі Array98

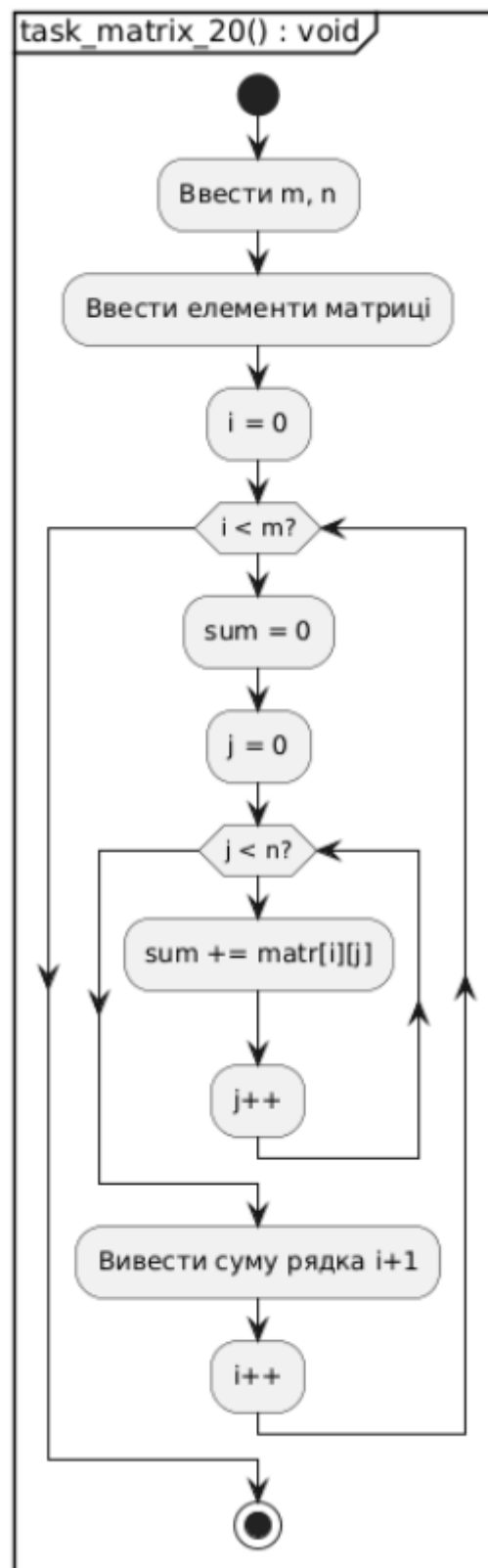


Рисунок 2 – Діаграма активності для задачі Matrix20

ДОДАТОК Г

Діалог з ШІ для самоаналізу

"Реалізація алгоритмів сортування та робота з файлами на мові C++" ось тема

Добре 🐼 Нова тема:

📘 «Реалізація алгоритмів сортування та робота з файлами мовою C++»

Робимо перевірку знань у звичному форматі.

♦ Частина 1. Тестові питання (1–6)

1. Яка бібліотека використовується для роботи з файлами у C++?

- | | |
|----|--------------------------------|
| A) | <code><algorithm></code> |
| B) | <code><fstream></code> |
| C) | <code><vector></code> |
| D) | <code><string></code> |

2. Який клас використовується для запису у файл?

- | | |
|----|-------------------------|
| A) | <code>ifstream</code> |
| B) | <code>ofstream</code> |
| C) | <code>fstream_in</code> |
| D) | <code>filewriter</code> |

3. Який клас використовується для читання з файлу?

- | | |
|----|-----------------------|
| A) | <code>ifstream</code> |
| B) | <code>ofstream</code> |
| C) | <code>readfile</code> |
| D) | <code>scanner</code> |

4. Яка складність бульбашкового сортування у середньому випадку?

- | | |
|----|---------------|
| A) | $O(\log n)$ |
| B) | $O(n)$ |
| C) | $O(n^2)$ |
| D) | $O(n \log n)$ |

5. Який алгоритм сортування зазвичай швидший на практиці для великих масивів?

A)	Bubble	Sort
B)	Selection	Sort
C)	Quick	Sort
D)	Linear Search	

6. Що потрібно зробити після завершення роботи з файлом?

A)	Видалити	файл
B)	Закрити	файл
C)	Перезаписати	файл
D)	Очистити буфер вручну завжди	

♦ Частина 2. Відкриті питання (7–10)

7. Поясни алгоритм бульбашкового сортування.

8. У чому різниця між ifstream, ofstream і fstream?

9. Як перевірити, чи файл успішно відкрився?

10. Які помилки можуть виникати при роботі з файлами?

♦ Частина 3. Міні-практика

11. Напиши фрагмент коду C++, який:

- ✓ зчитує числа з файлу
 - ✓ сортує їх за зростанням
 - ✓ записує результат у інший файл
- (достатньо короткого фрагмента, не обов'язково повна програма)

Відповідай як завжди:

1	—	?
2	—	?
...		

Після цього зроблю перевірку + оцінку 🐼

♦ Частина 1. Тестові питання (1–6) 1 – В 2 – В 3 – А 4 – С 5 – С 6 – В ♦ Частина 2. Відкриті питання (7–10) 7. Поясни алгоритм бульбашкового сортування.

Бульбашкове сортування працює шляхом багаторазового проходу по масиву. На кожному проході порівнюються сусідні елементи, і якщо вони стоять у неправильному порядку, то міняються місцями. Після кожного проходу найбільший елемент переміщується в кінець масиву. Процес повторюється, доки масив не стане відсортованим.

8. У чому різниця між `ifstream`, `ofstream` і `fstream`?
`ifstream` використовується для читання даних із файлу. `ofstream` використовується для запису даних у файл. `fstream` дозволяє одночасно читати і записувати дані у файл.

9. Як перевірити, чи файл успішно відкрився? Після відкриття файлу потрібно перевірити стан потоку за допомогою методу `is_open()` або перевіркою самого об'єкта файлу в умові. Якщо файл не відкрився, програма має повідомити про помилку.

10. Які помилки можуть виникати при роботі з файлами? файл не існує; немає доступу до файлу; неправильний шлях до файлу; помилки формату даних під час читання; забутий виклик закриття файлу.

Добре



Перевіряємо

тему

«Реалізація алгоритмів сортування та робота з файлами мовою C++»

♦ Частина 1. Тестові питання

№ Відповідь Результат Коментар

1	В	☒	<fstream>
2	В	☒	ofstream для запису
3	А	☒	ifstream для читання
4	С	☒	Bubble Sort = $O(n^2)$
5	С	☒	Quick Sort
6	В	☒	Файл треба закривати

Результат: 6 / 6

Без помилок.

♦ Частина 2. Відкриті питання

▣ Бульбашкове сортування

✓ Правильно описано:

- багаторазові проходи
- порівняння сусідніх елементів

- обмін
- найбільший “спливає” вправо

Повна

Відповідь.

1 / 1

ifstream, ofstream, fstream

✓ Усе правильно:

- ifstream → читання
- ofstream → запис
- fstream → читання + запис

Коротко

і

Точно.

1 / 1

Перевірка відкриття файлу

✓ Правильно:

- is_open()
- перевірка стану потоку

Наприклад:

```
if (!file.is_open())
```

Все

коректно.

1 / 1

Помилки роботи з файлами

✓ Правильно названі:

- файл не знайдено
- доступ заборонений
- неправильний шлях
- формат даних
- забутий close()

Хороший

Список.

1 / 1

Підсумок

Частина Бал

Тести 6 / 6

Відкриті 4 / 4

 Фінальна оцінка: 10 / 10

Коментар викладача

Ти добре розумієш:

- алгоритми сортування
- файлові потоки
- типові помилки файлового вводу/виводу
- базову складність алгоритмів

Відповіді:

- структуровані
- без води
- технічно коректні.

Тобто по цій темі ти теж **готовий до захисту**.

 Раз уже ця тема пішла добре, ось типове складніше питання, яке люблять ставити:

Чому при роботі з файлами бажано перевіряти не тільки відкриття файлу, а й стан потоку після читання даних?

Можеш відповісти — і перевіримо глибше знання теми.

Чим відрізняється текстовий файл від бінарного при роботі в C++?

При роботі в C++ текстові й бінарні файли відрізняються **способом зберігання даних, читанням і записом**.

◇ 1. Текстовий файл

Текстовий файл зберігає дані у вигляді **символів**.

Наприклад файл може містити:

123

45

78

hello

Тобто числа записуються як послідовність символів:

- 123 = символ '1', '2', '3'

а не як внутрішнє двійкове представлення числа.

🔗 Особливості текстового файлу

☑ Людина може відкрити файл у:

- Notepad
- Visual Studio Code

і прочитати його.

Читання/запис:

ofstream	fout("data.txt");
fout << 123;	
ifstream	fin("data.txt");
fin >> x;	

Плюси

- читабельність
- зручне налагодження
- переносимість

Мінуси

- більший розмір файлу
 - повільніше читання/запис
 - потрібне перетворення текст ↔ число
-

♦ 2. Бінарний файл

Бінарний файл зберігає дані у вигляді **байтів**, тобто так, як вони реально лежать у пам'яті.

Наприклад число:

```
int x = 123;
```

записується як 4 байти (для int).

Запис

ofstream	fout("data.bin",	ios::binary);
fout.write((char*)&x, sizeof(x));		

Читання

```
ifstream                                fin("data.bin",                ios::binary);
fin.read((char*)&x, sizeof(x));
```

Особливості

✗ Людина не може нормально прочитати такий файл у текстовому редакторі. Відкриєш — побачиш “кашу”.

Плюси

- менший розмір
- швидший ввід/вивід
- без конвертації типів

Мінуси

- нечитабельність
 - залежність від архітектури:
 - розмір типів
 - порядок байтів
-

Порівняння

Характеристика Текстовий Бінарний

Зберігання	символи	байти
Читабельність	☑	✗
Швидкість	повільніше	швидше
Розмір	більший	менший
Відлагодження	зручне	складніше

Коротка відповідь для захисту

Текстовий файл зберігає дані у вигляді символів, тому його можна прочитати людиною. Бінарний файл зберігає байти у внутрішньому форматі даних, що забезпечує менший розмір і швидший доступ, але файл стає нечитабельним для людини.

Приклад:

- .txt → текстовий

- .bin, .dat → часто бінарні.

Це хороше питання, бо перевіряє чи ти розумієш не просто ifstream/ofstream, а **реальну модель зберігання даних в C++**.

У яких випадках бульбашкове сортування все ще доцільно використовувати?

Бульбашкове сортування (**Bubble Sort**) у C++ загалом неефективне для великих масивів через складність **$O(n^2)$** , тому для серйозних задач його майже не використовують. Але є ситуації, де воно все ще має сенс.

♦ 1. Навчання алгоритмам

Найчастіший випадок.

Bubble Sort добре показує базові поняття:

- проходи по масиву
- порівняння елементів
- обмін значень
- вкладені цикли

Приклад:

```
for(int i = 0; i < n - 1; i++)
    for(int j = 0; j < n - 1 - i; j++)
        if(a[j] > a[j+1])
            swap(a[j], a[j+1]);
```

Через простоту його часто дають на перших лабораторних.

♦ 2. Дуже маленькі масиви

Якщо масив дуже малий:

$n \leq 10$ або 20

різниця між Bubble Sort і складнішими алгоритмами майже непомітна.

У такому випадку:

- простота коду може бути важливішою за продуктивність.

♦ 3. Майже відсортовані дані

Якщо додати прапорець обміну:

```
bool swapped;
```

то алгоритм може завершитися раніше.

if (!swapped) break;

Тоді:

- найкращий випадок $\rightarrow O(n)$

Це корисно, якщо масив уже майже відсортований.

♦ 4. Коли важлива простота реалізації

У простих навчальних або тестових програмах.

Наприклад:

- демонстрація сортування
- швидкий чернетковий код
- перевірка логіки роботи з масивами

♦ 5. Обмежене середовище

У дуже простих системах або мікроконтролерах інколи обирають максимально простий алгоритм.

Хоча на практиці навіть там часто є кращі варіанти.

✗ Коли НЕ варто використовувати

Bubble Sort поганий для:

- великих масивів
- продуктивних програм
- роботи з файлами великих обсягів даних

Там краще:

- `std::sort()`
- Quick Sort
- Merge Sort
- Heap Sort

Коротке порівняння

Ситуація	Bubble Sort
Навчання	☒
Маленькі масиви	☒
Майже відсортований масив	☒

Ситуація	Bubble Sort
Великі дані	✗
Продуктивні програми	✗

🎓 Коротка відповідь для захисту

Бульбашкове сортування доцільно використовувати для навчання, для невеликих масивів або майже відсортованих даних. Для великих наборів даних воно неефективне через квадратичну складність $O(n^2)$.

Тобто Bubble Sort — це як навчальний велосипед: для Tour de France не підійде, але щоб зрозуміти, як взагалі крутяться колеса, цілком годиться 😊